ADOPT-02: Platform Health Assessment

> **Series:** ADOPT — Observability Adoption & Maturity | **Notebook:** 2 of 6 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Overview

A healthy observability platform is the foundation for everything else — alerting, troubleshooting, capacity planning, and automation all depend on complete and accurate data. This notebook provides a structured approach to assessing Dynatrace platform health: OneAgent deployment coverage, data ingestion rates, entity monitoring completeness, ActiveGate status, and license consumption. The result is a platform health scorecard you can review regularly.

Table of Contents

1. OneAgent Deployment Coverage
2. Host Monitoring Health
3. Service and Process Discovery
4. Data Ingestion Rates
5. ActiveGate Health
6. License and Consumption Tracking
7. Building a Platform Health Scorecard
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS (Grail) on the Dynatrace Platform Subscription (DPS) for § 1.2 and § 6. The DQL does not apply to Dynatrace Managed, which has no Grail.
Permissions	`storage:entities:read`, `storage:smartscape:read`, `storage:metrics:read`, `storage:logs:read`, `storage:spans:read`, `storage:system:read`
Data	At least 24 hours of OneAgent data
Audience	Platform engineers, Dynatrace administrators

1. OneAgent Deployment Coverage

OneAgent coverage is the most fundamental health indicator. Gaps in deployment mean gaps in visibility. The goal is to understand how many hosts are monitored and whether any are running outdated agent versions.

1.1 Total Monitored Hosts

```
// Count hosts seen in the last 24 hours
fetch dt.entity.host, from:-24h
| summarize total_hosts = count()

// Without from:, dt.entity.* returns only entities seen in the default two-hour window — on the
// validation tenant that was 15 hosts against 33 over 24 hours. Every inventory query in this
// notebook uses the same from:-24h so the counts agree with each other.
//
// Smartscape equivalent: smartscapeNodes "HOST", from:-24h | summarize total_hosts = count()
// The two surfaces can report different counts (FAQ-16); for an inventory, keep the classic query.
```

1.2 Hosts by Monitoring Mode

Dynatrace supports multiple monitoring modes. Full-Stack provides the deepest visibility; Infrastructure and Foundation & Discovery collect progressively less.

Read the mode from billing, not from the host entity. The classic `monitoringMode` property on `dt.entity.host` is not populated for every host. On the validation tenant (10/01/2026), 29 of the 36 hosts billed as Full-Stack in the last 24 hours had `monitoringMode` empty, so a `by:{monitoringMode}` breakdown put four in five Full-Stack hosts in a null bucket. On DPS, each monitored host emits billing usage events that name the capability it was charged under — that is the mode that counts.

> **SaaS 1.347 — staged tenant rollout from 09/16/2026:** billing usage events are moving their host ID from `dt.entity.host` to `dt.smartscape.host`. Verbatim: *"If you use custom DQL queries that reference entity ID attributes in billing usage events, review and update them to the new Smartscape attribute names."* The query below reads `coalesce(toString(dt.smartscape.host), dt.entity.host)`, so it returns the same hosts before and after the change reaches your tenant (on the validation tenant both fields were written, with identical values, on 10/05/2026). What's new in SaaS 1.347 (DT docs).

```
// Hosts by the capability they were billed under (DPS), last 24 hours.
// A host can appear in more than one row — on the validation tenant 19 of 39 hosts did,
// typically Full-Stack plus Code Monitoring. Non-host capabilities have no host ID
// and are filtered out.
fetch dt.system.events, from:-24h
| filter event.kind == "BILLING_USAGE_EVENT"
// SaaS 1.347 moves the host ID to dt.smartscape.host; coalesce reads whichever field your tenant writes.
| fieldsAdd host = coalesce(toString(dt.smartscape.host), dt.entity.host)
| filter isNotNull(host)
| summarize {hosts = countDistinctExact(host)}, by:{billed_as = event.type}
| sort hosts desc
```

1.3 Agent Version Distribution

Running outdated OneAgent versions can lead to missed features, security vulnerabilities, and compatibility issues. This query identifies the distribution of agent versions across your environment.

```
// Identify OneAgent version distribution across hosts seen in the last 24
hours.
// The host property is installerVersion (e.g. 1.343.90.20260806-093919);
agentVersion exists
// only on dt.entity.process_group_instance and fails here with
FIELD_DOES_NOT_EXIST.
fetch dt.entity.host, from:-24h
| summarize {host_count = count()}, by:{installerVersion}
| sort host_count desc
| limit 20

// No Smartscape equivalent: the OneAgent version is not a Smartscape HOST
node field.
```

> **Tip — what "current" means for OneAgent.** Every OneAgent is version 1.x, so "major version" says nothing; what matters is the release (1.343, 1.345, …) and whether it is still supported. Dynatrace supports a OneAgent or ActiveGate release for **9 months (Standard Support) or 12 months (Enterprise Support) from its release date**. Treat any host on a release older than that window as out of support, and use FAQ-04 to choose between auto-update and coordinated update windows.

2. Host Monitoring Health

Beyond simple counts, we need to verify that monitored hosts are actively reporting data. A host entity may exist but stop sending metrics if the agent crashes or the host is decommissioned.

2.1 Host CPU Utilization Overview

This query provides a quick health check — if hosts are reporting CPU metrics, the agent is functional. PaaS hosts (`paasVendorType` set — Kubernetes application-only, Fargate)

do not report `dt.host.*` metrics, so they are absent from these two queries (16 of 23 hosts on the validation tenant, 10/23/2026).

```
// Top 10 hosts by average CPU usage over last 1 hour
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

2.2 Host Memory Utilization

```
// Top 10 hosts by average memory usage over last 1 hour
timeseries avgMem = avg(dt.host.memory.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgMemValue = arrayAvg(avgMem)
| sort avgMemValue desc
| limit 10
```

3. Service and Process Discovery

Auto-discovered services and process groups indicate the depth of application-level monitoring. A healthy deployment should show services that match your known application inventory.

3.1 Service Count and Types

```
// Count services by service type
fetch dt.entity.service, from:-24h
| summarize {service_count = count()}, by:{serviceType}
| sort service_count desc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | summarize service_count = count(), by:{dt.service.sdv1_type}
//   | sort service_count desc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration discovery inventory keep the
classic query above.
// Field maps: serviceType -&gt; dt.service.sdv1_type.
// dt.service.sdv1_type is set only when dt.service_detection.version is 1; on
SDv2 services it is null — add dt.service_detection.version to the by: list to
see which applies.
```

3.2 Process Group Inventory

```
// Count process groups by technology type (a process group can run several)
fetch dt.entity.process_group, from:-24h
| expand tech = softwareTechnologies
| parse tech, "'type:' LD:technology ','"
| summarize {pg_count = countDistinctExact(id)}, by:{technology}
| sort pg_count desc
| limit 15

// Smartscape note (dt.entity.* is deprecated but still functional):
Smartscape models
// individual processes, not process GROUPS — smartscapeNodes "PROCESS" is a
different
// granularity (process instances), so its results are not comparable to a
process-group
// query. Keep the classic dt.entity.process_group query above.
```

4. Data Ingestion Rates

Understanding how much data flows into Dynatrace helps with capacity planning, cost management, and identifying gaps. A sudden drop in ingestion often signals an agent

outage or configuration change.

4.1 Log Ingestion Volume Over Time

```
// Log ingestion trend over the last 24 hours by hour
// Whole hours only — a relative from:-24h gives a partial first and last
hour.
fetch logs, from:now()@h - 24h, to:now()@h
| makeTimeseries log_count = count(), interval:1h
```

4.2 Log Ingestion by Source

```
// Top 10 log sources by volume in the last 24 hours
fetch logs, from:-24h
| summarize log_count = count(), by:{log.source}
| sort log_count desc
| limit 10
```

4.3 Span Ingestion Volume

```
// Span ingestion trend over the last 24 hours
// Whole hours only — a relative from:-24h gives a partial first and last
hour.
fetch spans, from:now()@h - 24h, to:now()@h
| makeTimeseries span_count = count(), interval:1h
```

5. ActiveGate Health

ActiveGates serve as routing, monitoring extension, and API endpoints. Their health directly impacts data collection reliability.

5.1 ActiveGate Inventory

```
// List all ActiveGates with their properties
smartscapeNodes "ACTIVEGATE"
| fields name, version = dt.active_gate.version, group =
dt.active_gate.group.name, zone = dt.network_zone.id, is_containerized,
modules
| sort name asc

// ActiveGates are read from the Smartscape ACTIVEGATE node. There is no
classic DQL entity type for
// ActiveGates in any spelling (active_gate, environment_active_gate,
environment_activegate):
// `fetch dt.entity.active_gate` returns zero rows on every tenant, which
looks exactly like
// "no ActiveGates deployed".
// ActiveGate 1.343 deprecates GET /api/v2/activeGates,
/api/v2/activeGates/{agId} and
// /api/v2/activeGates/groups: "These are replaced by the ACTIVEGATE
Smartscape node."
```

5.2 ActiveGate Metric Health

Self-monitoring metrics confirm ActiveGates are processing data. A flat or zero metric suggests the ActiveGate is unhealthy.

```
// Connected agent modules per ActiveGate over the last 1 hour.
// A healthy routing ActiveGate holds a steady, non-zero count; a flat zero
means agents are not
// reaching it, and a sudden drop means they stopped.
// dt.active_gate.id is the same identity the ACTIVEGATE Smartscape node
carries, so the ids
// (0xd54e5d57, …) resolve to names via smartscapeNodes "ACTIVEGATE".
//
// Discover self-monitoring keys rather than trusting a remembered name —
timeseries against a key
// that does not exist returns an empty result, not an error:
//   metrics from:now()-2h | filter startsWith(metric.key,
"dt.sfm.active_gate") | fields metric.key
// (`metrics` takes from: with no leading comma; `metrics, from:` is a
PARSE_ERROR.)
timeseries connected =
avg(dt.sfm.active_gate.communication.agent_modules.connected), from:-1h, by:
{dt.active_gate.id}
```

6. License and Consumption Tracking

Tracking license consumption helps prevent unexpected overages and ensures you are getting value from your investment.

6.1 Host Monitoring Consumption

A host count is not a consumption estimate. On DPS, Full-Stack Monitoring is measured in **GiB-hours** — host memory multiplied by hours monitored — and Infrastructure Monitoring in **host-hours**. A 64 GiB host and a 4 GiB host are one host each, but under Full-Stack one consumes sixteen times the other. Read consumption from the billing usage events instead. (On classic host-unit licensing, host units also scale with memory, so the same caution applies.) FINOPS-01 covers every capability's unit field.

```
// Full-Stack consumption per host over the last 7 days, in GiB-hours
fetch dt.system.events, from:-7d
| filter event.kind == "BILLING_USAGE_EVENT" and event.type == "Full-Stack
Monitoring"
// Host ID read as in § 1.2 — SaaS 1.347 moves it from dt.entity.host to
dt.smartscape.host.
| fieldsAdd host = coalesce(toString(dt.smartscape.host), dt.entity.host)
| summarize {gib_hours = sum(billed_gibibyte_hours)}, by:{host}
| sort gib_hours desc
| limit 20
```

6.2 Log Ingest Volume for Cost Awareness

Log ingest is billed by volume, not by record count, so read it from the billing events in GiB.

```
// Billed log ingest per day over the last 7 days, in GiB
// Whole days only — a relative from:-7d gives a partial first and last day.
fetch dt.system.events, from:now()@d - 7d, to:now()@d
| filter event.kind == "BILLING_USAGE_EVENT"
| filter event.type == "Log Management &amp;amp; Analytics - Ingest &amp;amp; Process"
| summarize {billed_gib = sum(billed_bytes) / 1073741824.0}, by:{day =
bin(timestamp, 24h)}
| sort day asc
```

7. Building a Platform Health Scorecard

Combine the results from the queries above into a regular health scorecard. Review this scorecard weekly or monthly to track platform health trends. The targets are community starting points, not Dynatrace-published thresholds — set your own once you have a baseline.

Recommended Scorecard Metrics

Metric	Target	How to Measure
Host Coverage	> 95% of known hosts monitored	Compare entity count vs CMDB

Metric	Target	How to Measure
Agent Version Currency	Every host on a OneAgent release still inside its support window	Version distribution query (§ 1.3)
Service Discovery	All known services auto-detected	Compare service count vs app inventory
Log Ingestion Stability	< 10% daily variance	Billed log ingest per day (§ 6.2)
Span Ingestion Active	> 0 spans per hour	Span count query
ActiveGate Health	All AGs reporting metrics	AG connection metrics
Dynatrace Intelligence Active	Problems detected in last 7 days	Problem count query
Alert Quality	Short-lived problem share flat or falling	Alerting quality check from ADOPT-01 § 7.4

Scoring Guide

Score	Status	Action
8/8 metrics green	Healthy	Continue regular monitoring
6-7 green	Minor gaps	Address within current sprint
4-5 green	Significant gaps	Prioritize remediation
< 4 green	Critical	Escalate to platform team

8. Summary and Next Steps

Key Takeaways

- Platform health assessment should be a regular practice, not a one-time exercise
- OneAgent coverage and data ingestion stability are the two most critical health indicators
- A simple scorecard with 8-10 metrics provides actionable visibility into platform health

- Gaps discovered here directly inform your maturity roadmap from ADOPT-01

Next Steps

- Proceed to **ADOPT-03: Success Metrics** to define MTTR, MTTD, and other outcome-based metrics
- Schedule a recurring health scorecard review (weekly recommended for new deployments)
- Compare discovered entities against your CMDB to identify coverage gaps

References

- Full-Stack Monitoring (DT docs) — *"Dynatrace uses GiB-hours (referred to as "memory-gibibyte-hours" in your rate card) as the unit of measure"*
- Infrastructure Monitoring (DT docs) — *"Infrastructure Monitoring consumption is measured in host hours"*
- Support policy (Dynatrace) — OneAgent and ActiveGate releases: 9 months Standard Support, 12 months Enterprise Support, from the release date
- ActiveGate 1.343 release notes (DT docs) — *"These are replaced by the ACTIVEGATE Smartscape node."*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.